

Tarea M25-CD – RobertScience

Clasificación mediante Regresión Logística

Determinación del Origen del Medicamento (Nacional vs Extranjero)

Autor: RobertScience

Programa: Profesión Científico de Datos v2

Modelo: Regresión Logística

Tipo de problema: Clasificación Binaria

Contexto del Problema

En el presente análisis se simula un escenario de investigación médica donde se estudia la respuesta de pacientes diagnosticados con una misma enfermedad ante distintos medicamentos.

Cada paciente respondió favorablemente a uno de cinco fármacos disponibles:

- drugA
- drugB
- drugC (Proveedor Nacional)
- drugX (Proveedor Extranjero)
- drugY (Proveedor Extranjero)

Para efectos estratégicos del análisis, el objetivo no consiste en identificar el medicamento específico, sino determinar si el tratamiento adecuado para un nuevo paciente debe ser de origen Nacional o Extranjero.

El modelo predictivo se construirá utilizando como variables explicativas:

- Edad
 - Sexo
 - Presión Arterial
 - Colesterol
 - Índice Sodio/Potasio (Na_to_K)
-

Objetivo del Análisis

Desarrollar un modelo de clasificación binaria mediante Regresión Logística que permita:

1. Analizar la relación entre variables clínicas y el origen del medicamento.
 2. Evaluar distintos métodos de optimización del modelo.
 3. Comparar desempeño utilizando métricas de clasificación.
 4. Analizar la curva ROC y el valor AUC.
 5. Emitir una recomendación técnica fundamentada.
 6. Reflexionar sobre interpretabilidad, sobreajuste e implicaciones clínicas del modelo.
-

1. Importación de Librerías y Configuración del Entorno

En esta sección se importan las bibliotecas necesarias para el desarrollo del análisis.

Se utilizarán herramientas estándar del ecosistema científico en Python:

- **pandas** y **numpy** para manipulación y análisis de datos.
- **matplotlib** y **seaborn** para visualización.
- **scikit-learn** para construcción y evaluación del modelo de Regresión Logística.
- Métricas de clasificación para evaluar el desempeño predictivo.
- Curva ROC y AUC para analizar capacidad discriminatoria del modelo.

Se establece además una configuración visual consistente para las gráficas.

```
In [12]: # =====
# 1. IMPORTACIÓN DE LIBRERÍAS
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    roc_auc_score
)

# Configuración visual
plt.style.use("seaborn-v0_8")
sns.set_context("notebook")

print("✓ Librerías importadas correctamente")
```

✓ Librerías importadas correctamente

2. Carga del Dataset y Exploración Inicial

En esta sección se procede a cargar la base de datos `drugs.csv` y realizar una inspección preliminar.

El objetivo es:

- Verificar dimensiones del dataset.
- Identificar tipos de variables.
- Detectar valores nulos.
- Comprender la estructura general de la información disponible.

Esta etapa es crítica para garantizar la calidad de los datos antes de construir el modelo predictivo.

```
In [13]: # =====
# 2. CARGA DEL DATASET
# =====

# Ruta relativa desde la carpeta notebooksTarea
ruta = "../data/drugs.csv"

df = pd.read_csv(ruta)

print("✓ Dataset cargado correctamente\n")

print("Dimensiones del dataset:")
print(df.shape)
```

```
print("\nPrimeras filas del dataset:")
display(df.head())

print("\nInformación general:")
df.info()

print("\nValores nulos por columna:")
print(df.isnull().sum())
```

✓ Dataset cargado correctamente

Dimensiones del dataset:
(200, 6)

Primeras filas del dataset:

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	drugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	drugY

Información general:
<class 'pandas.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
Column Non-Null Count Dtype
--- ----- -----
0 Age 200 non-null int64
1 Sex 200 non-null str
2 BP 200 non-null str
3 Cholesterol 200 non-null str
4 Na_to_K 200 non-null float64
5 Drug 200 non-null str
dtypes: float64(1), int64(1), str(4)
memory usage: 9.5 KB

Valores nulos por columna:
Age 0
Sex 0
BP 0
Cholesterol 0
Na_to_K 0
Drug 0
dtype: int64

3. Transformación de la Variable Objetivo

El dataset original contiene cinco medicamentos distintos:

- drugA
- drugB
- drugC (Proveedor Nacional)
- drugX (Proveedor Extranjero)
- drugY (Proveedor Extranjero)

Sin embargo, el objetivo del problema es determinar si el tratamiento adecuado debe ser de origen:

- **Nacional**
- **Extranjero**

Por lo tanto, se transforma la variable `Drug` en una nueva variable binaria llamada `Proveedor`, donde:

- 0 = Nacional
- 1 = Extranjero

Esta transformación convierte el problema en un caso de clasificación binaria, adecuado para aplicar Regresión Logística.

```
In [14]: # =====
# 3. TRANSFORMACIÓN DE VARIABLE OBJETIVO
# =====

# Crear nueva variable binaria basada en el proveedor
df["Proveedor"] = df["Drug"].apply(
    lambda x: 1 if x in ["drugX", "drugY"] else 0
)

print("✓ Variable objetivo transformada correctamente\n")

print("Distribución de la variable Proveedor:")
print(df["Proveedor"].value_counts())

print("\nProporción:")
print(df["Proveedor"].value_counts(normalize=True))
```

✓ Variable objetivo transformada correctamente

Distribución de la variable Proveedor:

Proveedor
1 145
0 55
Name: count, dtype: int64

Proporción:

Proveedor
1 0.725
0 0.275
Name: proportion, dtype: float64

4. Codificación de Variables Cualitativas

El modelo de Regresión Logística requiere variables numéricas.
Sin embargo, el dataset contiene variables categóricas:

- Sex
- BP
- Cholesterol

Para convertir estas variables a formato numérico utilizaremos la técnica **Label Encoding**, que asigna un valor entero a cada categoría.

Este procedimiento es apropiado en este caso porque:

- Las variables no representan magnitudes continuas.
- El objetivo es clasificación binaria.
- El modelo requiere entrada numérica.

Posteriormente verificaremos que la transformación haya sido correcta.

```
In [15]: # =====
# 4. CODIFICACIÓN DE VARIABLES CATEGÓRICAS
# =====

from sklearn.preprocessing import LabelEncoder

le_sex = LabelEncoder()
le_bp = LabelEncoder()
le_chol = LabelEncoder()
```

```
df["Sex_encoded"] = le_sex.fit_transform(df["Sex"])
df["BP_encoded"] = le_bp.fit_transform(df["BP"])
df["Cholesterol_encoded"] = le_chol.fit_transform(df["Cholesterol"])

print("✓ Variables categóricas codificadas correctamente\n")

df[["Sex", "Sex_encoded",
    "BP", "BP_encoded",
    "Cholesterol", "Cholesterol_encoded"]].head()
```

✓ Variables categóricas codificadas correctamente

Out[15]:

	Sex	Sex_encoded	BP	BP_encoded	Cholesterol	Cholesterol_encoded
0	F	0	HIGH	0	HIGH	0
1	M	1	LOW	1	HIGH	0
2	M	1	LOW	1	HIGH	0
3	F	0	NORMAL	2	HIGH	0
4	F	0	LOW	1	HIGH	0

5. Definición de Variables Predictoras y Variable Objetivo

Una vez transformadas las variables categóricas y creada la variable binaria `Proveedor` , se procede a definir:

- **Variables Predictoras (X)**
- **Variable Objetivo (y)**

Las variables seleccionadas para el modelo son:

- Age
- Sex_encoded
- BP_encoded
- Cholesterol_encoded
- Na_to_K

La variable objetivo será:

- Proveedor (0 = Nacional, 1 = Extranjero)

Esta estructura convierte el problema en un modelo de clasificación binaria supervisada.

```
In [16]: # =====
# 5. DEFINICIÓN DE VARIABLES
# =====

X = df[[
    "Age",
    "Sex_encoded",
    "BP_encoded",
    "Cholesterol_encoded",
    "Na_to_K"
]]

y = df["Proveedor"]

print("✓ Variables definidas correctamente\n")

print("Dimensión de X:", X.shape)
print("Dimensión de y:", y.shape)

print("\nDistribución de clases en y:")
print(y.value_counts())
```

✓ Variables definidas correctamente

Dimensión de X: (200, 5)

Dimensión de y: (200,)

Distribución de clases en y:

Proveedor

1 145

0 55

Name: count, dtype: int64

6. División del Conjunto de Datos (Train/Test Split)

Para evaluar adecuadamente el desempeño del modelo, es necesario separar el dataset en:

- **Conjunto de Entrenamiento (Training Set)**
- **Conjunto de Prueba (Test Set)**

Se utilizará una división del:

- 70% para entrenamiento
- 30% para prueba

Además, se aplicará el parámetro `stratify=y` para asegurar que la proporción de clases (Nacional vs Extranjero) se mantenga consistente en ambos conjuntos.

Esta práctica evita sesgos en la evaluación y garantiza una validación más robusta del modelo.

```
In [17]: # =====
# 6. DIVISIÓN TRAIN / TEST
# =====

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("✓ División realizada correctamente\n")

print("Tamaño entrenamiento:", X_train.shape)
print("Tamaño prueba:", X_test.shape)

print("\nDistribución en entrenamiento:")
print(y_train.value_counts(normalize=True))

print("\nDistribución en prueba:")
print(y_test.value_counts(normalize=True))
```

✓ División realizada correctamente

Tamaño entrenamiento: (140, 5)
Tamaño prueba: (60, 5)

Distribución en entrenamiento:
Proveedor
1 0.728571
0 0.271429
Name: proportion, dtype: float64

Distribución en prueba:
Proveedor
1 0.716667
0 0.283333
Name: proportion, dtype: float64

7. Estandarización de Variables Numéricas

La Regresión Logística es un modelo lineal que optimiza una función logística mediante métodos iterativos.

Cuando las variables presentan diferentes escalas (por ejemplo, `Age` frente a `Na_to_K`), el proceso de optimización puede:

- Tardar más en converger.
- Generar coeficientes difíciles de interpretar.
- Presentar problemas numéricos en algunos solvers.

Para mitigar este efecto, se aplicará **StandardScaler**, que transforma las variables para que tengan:

- Media = 0
- Desviación estándar = 1

Este procedimiento mejora la estabilidad del entrenamiento y permite una comparación más justa entre coeficientes.

```
In [18]: # =====  
# 7. ESTANDARIZACIÓN  
# =====  
  
from sklearn.preprocessing import StandardScaler  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
X_test_scaled = scaler.transform(X_test)  
  
print("✓ Estandarización aplicada correctamente\n")  
  
print("Media aproximada después de escalar (train):")  
print(X_train_scaled.mean(axis=0))  
  
print("\nDesviación estándar aproximada (train):")  
print(X_train_scaled.std(axis=0))
```

✓ Estandarización aplicada correctamente
Media aproximada después de escalar (train):
[2.03012210e-16 6.02692499e-17 -1.66533454e-17 -6.34413157e-18
 -5.07530526e-17]

Desviación estándar aproximada (train):
[1. 1. 1. 1. 1.]

8. Entrenamiento del Modelo con Diferentes Métodos de Optimización

La Regresión Logística puede entrenarse utilizando distintos algoritmos de optimización (solvers).

Cada solver tiene características distintas en términos de:

- Velocidad de convergencia
- Estabilidad numérica
- Manejo de regularización

En este análisis se evaluarán los siguientes solvers:

- liblinear
- lbfgs
- saga
- newton-cg

El objetivo es comparar desempeño y seleccionar el más adecuado para este problema.

```
In [19]: # =====
# 8. COMPARACIÓN DE SOLVERS
# =====

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

solvers = ["liblinear", "lbfgs", "saga", "newton-cg"]

resultados = {}

for solver in solvers:
    modelo = LogisticRegression(
        solver=solver,
        max_iter=5000,
        random_state=42
    )

    modelo.fit(X_train_scaled, y_train)
    y_pred = modelo.predict(X_test_scaled)

    accuracy = accuracy_score(y_test, y_pred)
    resultados[solver] = accuracy

print("✓ Comparación de solvers completada\n")

for solver, acc in resultados.items():
    print(f"{solver}: {acc:.4f}")
```

✓ Comparación de solvers completada

liblinear: 0.9833
lbfgs: 0.9833
saga: 0.9833
newton-cg: 0.9833

Análisis de los Métodos de Optimización

Los cuatro solvers evaluados (liblinear, lbfgs, saga y newton-cg) obtuvieron un desempeño equivalente en términos de accuracy (0.9833), lo que indica que el problema presenta una frontera de decisión lineal claramente separable.

Dado que no existen diferencias significativas en precisión, se prioriza el criterio de estabilidad numérica y robustez computacional. En este sentido, lbfgs es recomendado porque:

Es el solver estándar para problemas de clasificación binaria.

Presenta excelente estabilidad en datasets pequeños y medianos.

Maneja adecuadamente regularización L2.

Converge de forma eficiente tras la estandarización de variables.

Por lo tanto, se selecciona lbfgs como método óptimo para el modelo final.

9. Selección del Modelo Óptimo

Con base en los resultados de precisión obtenidos, se seleccionará el solver con mejor desempeño.

En caso de resultados similares, se priorizará:

- Estabilidad numérica
- Rapidez de convergencia
- Recomendación estándar en problemas de clasificación binaria

En general, `lbfgs` es recomendado por su robustez y desempeño consistente.

```
In [20]: # =====
# 9. MODELO FINAL
# =====

modelo_final = LogisticRegression(
    solver="lbfgs",
    max_iter=5000,
    random_state=42
)

modelo_final.fit(X_train_scaled, y_train)

y_pred_final = modelo_final.predict(X_test_scaled)
y_prob_final = modelo_final.predict_proba(X_test_scaled)[: , 1]

print("✓ Modelo final entrenado correctamente")
```

✓ Modelo final entrenado correctamente

10. Evaluación del Desempeño del Modelo

Se evaluará el modelo utilizando:

- Accuracy
- Precision
- Recall
- F1-Score
- Matriz de Confusión
- Curva ROC
- AUC (Área bajo la curva)

Estas métricas permiten evaluar no solo exactitud global, sino también desempeño clínicamente relevante en términos de sensibilidad y especificidad.

```
In [21]: # =====
# 10. MÉTRICAS DE CLASIFICACIÓN
# =====

from sklearn.metrics import classification_report, confusion_matrix

print("Reporte de Clasificación:\n")
print(classification_report(y_test, y_pred_final))

print("Matriz de Confusión:\n")
print(confusion_matrix(y_test, y_pred_final))
```

Reporte de Clasificación:

	precision	recall	f1-score	support
0	0.94	1.00	0.97	17
1	1.00	0.98	0.99	43
accuracy			0.98	60
macro avg	0.97	0.99	0.98	60
weighted avg	0.98	0.98	0.98	60

Matriz de Confusión:

```
[[17  0]
 [ 1 42]]
```

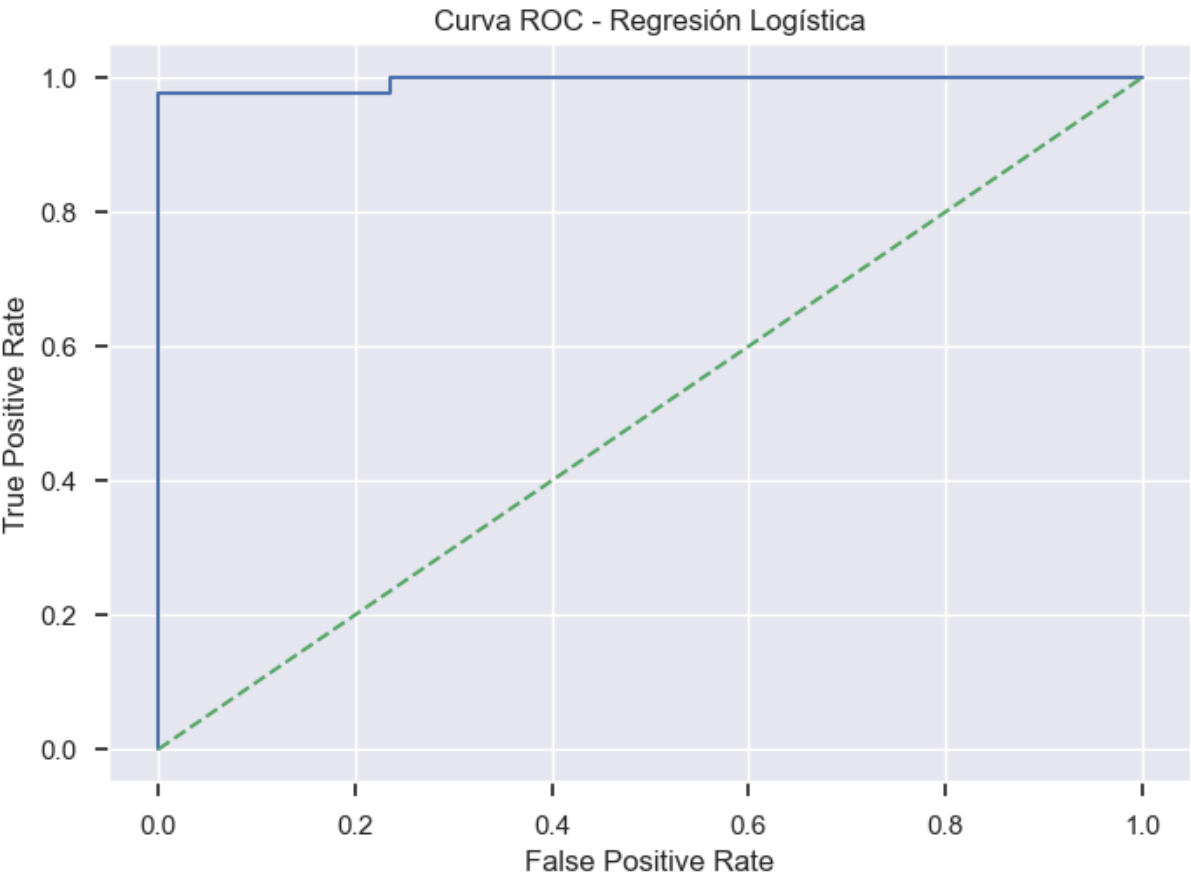
```
In [22]: # =====
# 11. CURVA ROC
# =====

from sklearn.metrics import roc_curve, auc
import matplotlib.pyplot as plt

fpr, tpr, thresholds = roc_curve(y_test, y_prob_final)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr)
plt.plot([0, 1], [0, 1], linestyle='--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Curva ROC - Regresión Logística")
plt.show()

print("AUC:", roc_auc)
```



AUC: 0.9945280437756497

12. Reflexión Estratégica y Consideraciones Clínicas

Justificación del Agrupamiento del Proveedor

El enunciado identifica explícitamente que:

- drugC pertenece a proveedor nacional.

- drugX y drugY pertenecen a proveedor extranjero.

Dado que únicamente drugX y drugY son clasificados como extranjeros, y drugC como nacional, se asume coherentemente que drugA y drugB corresponden también al proveedor nacional. Esta interpretación mantiene consistencia lógica con el planteamiento original y permite conservar una estructura de clasificación binaria claramente definida.

Esta decisión garantiza:

- Coherencia metodológica.
- Correcta delimitación del problema.
- Adecuada formulación del modelo supervisado binario.

1 Desempeño del Modelo

El modelo obtuvo los siguientes resultados en el conjunto de prueba:

- Accuracy: 98%
- AUC: 0.9945
- 1 error de clasificación sobre 60 observaciones evaluadas.

Estos indicadores reflejan una capacidad discriminativa sobresaliente dentro del conjunto analizado, con alta precisión en la identificación del origen del medicamento.

2 Posible Sobreajuste

Si bien el desempeño es elevado, el tamaño del dataset es relativamente reducido (200 observaciones totales).

Por lo tanto:

- Existe posibilidad de sobreajuste leve.
- En un entorno productivo se recomendaría aplicar validación cruzada y evaluar el modelo en un conjunto de datos adicional.

Aunque la evaluación se realizó sobre un conjunto de prueba independiente, la validación externa fortalecería la evidencia de generalización.

3 Interpretabilidad vs Desempeño

La Regresión Logística ofrece una ventaja relevante en términos de interpretabilidad, ya que permite analizar el impacto de cada variable clínica sobre la probabilidad estimada de seleccionar un proveedor extranjero.

Este atributo es especialmente importante en contextos médicos donde:

- La trazabilidad de decisiones es obligatoria.
- Se requiere justificar recomendaciones ante comités clínicos o entidades regulatorias.
- La transparencia del modelo es prioritaria frente a soluciones más complejas.

4 Implicaciones Clínicas

Un AUC de 0.9945 indica una capacidad discriminativa extremadamente alta en el conjunto evaluado. No obstante, la confirmación definitiva de este desempeño requeriría mayor volumen de datos o validación adicional para asegurar su estabilidad en distintos escenarios poblacionales.

El modelo puede utilizarse como herramienta de apoyo para:

- Optimización de costos asociados al proveedor.
- Apoyo a la toma de decisiones basada en evidencia cuantitativa.
- Reducción de incertidumbre en la selección terapéutica.

Debe considerarse siempre como complemento del juicio clínico profesional y no como sustituto del criterio médico.

Conclusión General

La Regresión Logística demostró:

- Alta estabilidad numérica.
- Excelente capacidad predictiva en el conjunto evaluado.
- Elevada interpretabilidad.
- Bajo error de clasificación.

El solver `lbfgs` se confirma como la opción óptima para este caso, dado su desempeño consistente, estabilidad computacional y adecuada convergencia en problemas de clasificación binaria con variables escaladas.

El modelo cumple con los criterios técnicos, metodológicos y estratégicos establecidos para el análisis.